Fundamentals of Natural Language Processing

Chapter 2 — Text Representation and Evaluation

Aymen Ben Brik

Chapter 2

Text Representation and Evaluation

A computer cannot operate on raw text. Before any model can "read" a sentence, the sentence must be *tokenized* (cut into discrete units), *numericalized* (converted into integer indices), and *vectorized* (embedded in a vector space). This chapter develops the chain rigorously: tokenization granularities, sparse and dense vector-space models, semantic similarity, and the evaluation metrics by which we compare NLP systems.

2.1 Tokenization fundamentals

2.1.1 Motivation

Tokenization is the unsung hero of every NLP pipeline. A poor tokenizer silently undermines the most sophisticated downstream model: out-of-vocabulary words become unknown tokens, morphological variants are treated as unrelated, multi-word expressions are fragmented. Conversely, a tokenizer aligned with the data distribution can shrink vocabulary size, accelerate training, and improve generalisation. Modern Large Language Models all rely on a single technical choice — *sub-word tokenization* — whose strengths and limitations every practitioner should understand.

2.1.2 Approach by problem: how many tokens are in this sentence?

Problem. Consider the sentence

> *"I've been to Sfax-University; it's amazing!"*

Count how many "tokens" it contains under each of the following naive policies:

1. split on whitespace;
2. split on whitespace and punctuation;
3. split into individual characters.

Solution sketch.

1. Whitespace only: {`I've`, `been`, `to`, `Sfax-University;`, `it's`, `amazing!`} — 6 tokens. Punctuation glued to words; "I've" is a single token.

2. Whitespace and punctuation: {`I`, `'ve`, `been`, `to`, `Sfax`, `-`, `University`, `;`, `it`, `'s`, `amazing`, `!`} — 12 tokens. The contraction *I've* is split, but so is the legitimate compound *Sfax-University*.

3. Characters: 35 tokens. Punctuation, capitalisation and spaces are all preserved, but every word is fragmented; the model would have to relearn "what is a word" from data.

Each policy entails a trade-off between vocabulary size and information preserved. There is no universally correct answer; the choice is engineering, guided by the downstream task and the language being modelled.

2.1.3 Definition and the tokenization pipeline

Definition 2.1.1 (Tokenization)**. Tokenization** is the process of converting a raw text string into an ordered sequence of discrete units called **tokens**, drawn from a finite inventory called the **vocabulary** V.

Definition 2.1.2 (Numericalization)**. Numericalization** is the bijective mapping $V \leftrightarrow \{0, 1, \ldots, |V| - 1\}$ that assigns each token an integer identifier.

The full pipeline is: *raw text* $\rightarrow$ tokens $\rightarrow$ token IDs $\rightarrow$ tensor inputs to a model. Every NLP system is built on top of these three trivially simple but practically critical steps.

2.1.4 Three levels of granularity

Level	Vocabulary size	Sequence length	OOV behaviour
Word	Very large (10^5–10^6)	Short	Unknown words become `[UNK]`
Character	Tiny (~ 100)	Very long	No OOV (any character is in V)
Sub-word	Moderate (10^4–10^5)	Moderate	Robust: any word splits into known pieces

Word tokenization. The intuitive default. Tokens are entire words separated by whitespace and punctuation rules. Strengths: each token carries lexical meaning, and downstream models converge faster on a small annotated dataset. Weaknesses: vocabulary explodes for morphologically rich languages (*Arabic, Turkish, Finnish*); typos and rare words become `[UNK]`; cannot generalise across morphological variants (*run / runs / running* are three unrelated tokens).

Character tokenization. Each character is a token. Strengths: vocabulary is bounded and stable across languages; no out-of-vocabulary problem (any unseen word becomes a sequence of known characters). Weaknesses: sequences become 5–10$\times$ longer, increasing both memory and compute; the model must learn from scratch which character sequences form meaningful units.

Sub-word tokenization. The dominant choice in 2020s NLP. Vocabulary contains *frequent words as wholes* and *rare words split into meaningful pieces*. The split is learnt from data using algorithms such as Byte-Pair Encoding (BPE), WordPiece (BERT) and SentencePiece (XLM-R, T5). Sub-word tokenization combines the strengths of word-level (semantic units for common words) and character-level (no OOV) approaches.

2.1.5 Pre-processing operations

Tokenization is often combined with ad-hoc transformations that modify or normalise the input string before splitting.

- **Lowercasing**: "Apple" and "apple" become equivalent. Useful for spelling-insensitive tasks but destroys signal in NER (a capitalised *Apple* is more likely the company).
- **Punctuation removal**: simplifies vocabulary but loses sentence boundaries.
- **Stop-word removal**: discard high-frequency words (*the, of, in*) that carry little task-specific information. Useful for keyword search; harmful for syntax-sensitive tasks.
- **Stemming**: chop morphological suffixes by hand-coded rules (Porter, Snowball). *running* $\to$ *run*. Fast but linguistically crude.
- **Lemmatization**: map a word to its dictionary form using a morphological analyser. *better* $\to$ *good*. Slower but linguistically principled.

Remark 2.1.1. Modern large language models perform little or no manual pre-processing: they are trained on raw text with sub-word tokenization, and the model itself learns whatever normalisations it needs. Heavy pre-processing is typical of statistical-era pipelines and remains useful for small-data, classical-ML systems.

2.1.6 Byte-Pair Encoding (BPE) in detail

BPE is the simplest sub-word algorithm and the workhorse of GPT-2, GPT-3, GPT-4, RoBERTa, and many others. It iteratively merges the most frequent pair of adjacent symbols.

Algorithm.

1. Start with a vocabulary that contains every individual character of the corpus.
2. Treat each word as a sequence of characters with an end-of-word marker.
3. Count the frequency of every adjacent symbol pair across the corpus.
4. Merge the most frequent pair into a new symbol; add it to the vocabulary.
5. Repeat steps 3–4 until a target vocabulary size is reached.

Example 2.1.1 (A miniature BPE run)**.** Suppose the corpus consists of the words `low`, `lower`, `newest`, `widest`, with frequencies $5, 2, 6, 3$. The initial vocabulary is {`l, o, w, e, r, n, s, t, i, d, </w>`}. Counting pairs:

$$\texttt{e s} = 6 + 3 = 9, \quad \texttt{s t} = 9, \quad \texttt{l o} = 7, \quad \texttt{o w} = 7, \ldots$$

The pair `e s` (9 occurrences) is merged first into a new symbol `es`. Recount, merge again — now `es t` (9 occurrences) into `est`. Continue: `lo`, `low`, `ne`, `new`, … After enough merges, frequent words like `low</w>` become single tokens, while rare words remain decomposed. A new word like `lowest` is then tokenized as `low + est`, even though it never appeared in training.

Why BPE works. The algorithm provides a natural rate–distortion trade-off. As vocabulary grows, the average sequence length shrinks, but the "unit cost" per token increases (each token carries more information). Empirically, vocabularies of 30k–100k sub-words give an excellent trade-off across many languages.

2.1.7 Multilingual and dialectal challenges

Tokenization choices that are adequate for English may fail for other languages.

- **Whitespace-free scripts.** Chinese, Japanese, Thai write without spaces. Word tokenization requires segmentation algorithms (e.g. Jieba, MeCab).
- **Morphologically rich languages.** Arabic and Turkish concatenate prefixes and suffixes onto a stem, generating very large surface-form inventories. Sub-word tokenization is essential.
- **Code-switching and dialects.** Tunisian Arabic frequently mixes Modern Standard Arabic, French, and Latin-script transliterations ("arabizi"). A general-purpose multilingual tokenizer (e.g. XLM-R's SentencePiece) is recommended over single-language tokenizers.
- **Right-to-left scripts.** Display order and logical order can differ; numerical IDs operate on logical order, but evaluation tools must be aware of the visual rendering.

2.1.8 Worked examples

Example 1 (Granularity choice — Easy). You are training a classifier on 1000 short SMS messages, mostly in English with frequent typos. Which tokenization granularity is most appropriate, and why?

Solution. Sub-word tokenization. Word-level would suffer from OOV (typos produce never-seen surface forms); character-level would lengthen sequences and dilute the signal in a small dataset. Sub-word splits typos into recognised pieces (`hapy` → `ha + py`) without exploding the vocabulary.

Example 2 (BPE iteration — Medium). Apply two iterations of BPE to the toy corpus {`aaab`, `aabb`, `abab`} (each of frequency 1). Show the resulting vocabulary after each merge.

Solution. Initial vocabulary: {`a`, `b`}.

Iteration 1: count pairs — `a a` appears in `aaab` (twice) and `aabb` (once) and `abab` (zero) = 3; `a b` appears in `aaab` (once) and `aabb` (once) and `abab` (twice) = 4; `b a` appears once (in `abab`); `b b` once. Most frequent: `a b` (4). Merge into `ab`. Vocabulary: {`a`, `b`, `ab`}. Re-tokenization: `aaab` → `a a ab`; `aabb` → `a ab b`; `abab` → `ab ab`.

Iteration 2: count new pairs — `a a` (in `aaab`, once) = 1; `a ab` (in `aaab` and `aabb`) = 2; `ab b` (in `aabb`) = 1; `ab ab` (in `abab`) = 1. Most frequent: `a ab` (2). Merge into `aab`. Vocabulary: {`a`, `b`, `ab`, `aab`}.

Example 3 (Tokenizer mismatch — Hard). A pretrained English BERT tokenizer applied to Tunisian Arabic in Latin script ("arabizi") splits the word `n7ebek` ("I love you") as `[n, ##7, ##e, ##bek]`. Discuss the consequences for a downstream classifier and propose two remediation strategies.

Solution.

- **Consequences.** The numeric digit 7 (which substitutes for the Arabic letter ʿ) is treated as foreign; the model sees an unfamiliar pattern with no semantic anchoring. The classifier's representation of the word will be near-random, and its predictions for sentences containing arabizi will be poor.
- **Remediations.**
  1. Use a multilingual tokenizer (e.g. `xlm-roberta-base`'s SentencePiece) trained on much broader data including Latin-script Arabic.
  2. Train a domain-specific tokenizer on a Tunisian-Arabic corpus before fine-tuning the model. The merges will then capture frequent dialectal sub-words natively.

2.1.9 Python snippet: comparing tokenizers

```python
# pip install nltk transformers
import nltk
from transformers import AutoTokenizer

text = "I've␣been␣to␣Sfax-University;␣it's␣amazing!"

# Word-level (NLTK)
nltk.download('punkt_tab', quiet=True)
print("Word␣tokens␣(NLTK):", nltk.word_tokenize(text))

# Sub-word (BERT WordPiece)
bert = AutoTokenizer.from_pretrained("bert-base-uncased")
print("BERT␣sub-word␣tokens:", bert.tokenize(text))

# Sub-word (GPT-2 byte-level BPE)
gpt2 = AutoTokenizer.from_pretrained("gpt2")
print("GPT-2␣BPE␣tokens:␣␣␣␣", gpt2.tokenize(text))
```

The same input produces different tokenisations under different algorithms. Reading the tokenized output is the first debugging step when a model behaves unexpectedly.

2.1.10 Exercises

1. For each scenario, recommend a tokenization granularity (word / character / sub-word) and justify briefly:
   (a) Building a spelling corrector for French SMS;
   (b) Training a sentiment classifier on 50 million Web reviews;
   (c) Building a name-entity recognizer for German legal documents.
2. Explain in one paragraph why character-level tokenization, despite its $|V| \approx 100$, has *not* replaced sub-word tokenization in modern LLMs.
3. Tokenize the sentence "*The CEOs' meeting was rescheduled.*" with two different policies (whitespace+punctuation, and a sub-word tokenizer of your choice). Comment on the differences for the words *CEOs'* and *rescheduled*.

4. Carry out three iterations of BPE on the toy corpus $\{\texttt{xay}, \texttt{xay}, \texttt{yax}, \texttt{xy}\}$ where each word has frequency 1. List the new vocabulary at each step.

5. (Synthesis.) For a multilingual Tunisian dialect dataset (mixing Arabic script, Latin-script arabizi, and French), discuss the trade-offs between (i) using a single multilingual sub-word tokenizer (e.g. XLM-R), (ii) training a domain-specific tokenizer on the dataset, and (iii) using language-specific tokenizers for each script and merging the outputs.

2.2 Sparse representations: Bag-of-Words and TF–IDF

2.2.1 Motivation

After tokenization, a text is a sequence of integer IDs. To feed such a sequence to a classical machine-learning algorithm — logistic regression, k-nearest-neighbours, support-vector machine — we need a *fixed-size vector* representation that does not depend on document length. The simplest such representation is the **Bag-of-Words** (BoW), which counts how often each vocabulary item appears. BoW and its weighted refinement **TF–IDF** dominated information retrieval and text classification for two decades; they remain strong baselines, fast to compute, easy to debug, and competitive on small or domain-specific data sets.

2.2.2 Approach by problem: are these two reviews similar?

Problem. Compare the following two hotel reviews:

- d_1: "*The room was clean. The room was quiet.*"
- d_2: "*The breakfast was great. The room was clean.*"

Propose a numerical representation for each that lets us measure their similarity, ignoring word order.

Solution sketch. Build a vocabulary by collecting all distinct words: $V = \{$the, room, was, clean, q (size 7). For each document, count how often each vocabulary word appears:

$$\mathbf{x}_{d_1} = (2, 2, 2, 1, 1, 0, 0)^\top, \quad \mathbf{x}_{d_2} = (2, 1, 2, 1, 0, 1, 1)^\top.$$

The two vectors share many components (e.g. both contain "the", "room", "was", "clean") and differ on others (*quiet* only in d_1, *breakfast / great* only in d_2). Their dot product gives a coarse measure of similarity. This embarrassingly simple recipe is the starting point of all classical NLP.

2.2.3 Vector space models

Definition 2.2.1 (Vector space model)**.** A **vector space model** (VSM) of a corpus is an embedding $\phi : \mathcal{T} \to \mathbb{R}^d$, where $\mathcal{T}$ is the set of textual objects (words or documents) and d is the embedding dimension. Documents are represented as vectors so that geometric operations (dot product, cosine similarity, distance) translate into linguistic operations (similarity, retrieval, clustering).

2.2.4 Bag-of-Words

Definition 2.2.2 (Bag-of-Words representation)**.** Fix a vocabulary $V = \{w_1, w_2, \ldots, w_{|V|}\}$. The **Bag-of-Words** (BoW) representation of a document d is the vector

$$\phi_{\text{BoW}}(d) = \big(\text{tf}(w_1, d),\ \text{tf}(w_2, d),\ \ldots,\ \text{tf}(w_{|V|}, d)\big) \in \mathbb{R}^{|V|},$$

where $\text{tf}(w, d)$ is the *term frequency* (raw count) of word w in d. The representation is named "bag" because the order of words is discarded: the document is treated as a multiset.

Alternative weightings of tf**.**

- **Boolean**: $\text{tf}(w, d) \in \{0, 1\}$, indicating presence.
- **Raw count**: $\text{tf}(w, d) =$ number of occurrences.
- **Log-normalized**: $\text{tf}(w, d) = \log(1 + \#(w, d))$, mitigating the bias toward long documents.
- **Length-normalized**: divide raw counts by document length so that all documents have ℓ_1-norm 1.

2.2.5 The document–term matrix

Definition 2.2.3 (Document–term matrix)**.** Given a corpus $\{d_1, \ldots, d_N\}$ and a vocabulary V, the **document–term matrix** is the matrix $X \in \mathbb{R}^{N \times |V|}$ whose entry $X_{ij} = \text{tf}(w_j, d_i)$.

In practice $|V|$ is in the tens or hundreds of thousands, while only a few dozen distinct words appear in each document. The matrix is therefore extremely *sparse*: a vast majority of entries are zero. Efficient storage formats (compressed sparse row, CSR; compressed sparse column, CSC) preserve memory by storing only non-zero entries.

2.2.6 Limitations of Bag-of-Words

- **No word order**: the documents "*Tunisia beat Egypt*" and "*Egypt beat Tunisia*" yield identical BoW vectors despite their opposite meanings.
- **Equal weight to all words**: a function word like "*the*" contributes the same magnitude as a domain-specific term like "*Paracetamol*", even though the latter is far more discriminative.
- **No semantic similarity**: "*cat*" and "*kitten*" are orthogonal vectors in $\mathbb{R}^{|V|}$ — as far apart as "*cat*" and "*democracy*".
- **Curse of dimensionality**: $|V| \approx 50{,}000$ produces vectors that are computationally cheap (sparse) but statistically noisy — many features contribute marginally to any specific task.

The first two limitations are tackled in this section by TF–IDF; the third requires dense embeddings (Section **??**). The first — the loss of order — is fundamentally unfixable in a bag-of-words framework and is a key driver of the move to recurrent and Transformer models.

2.2.7 TF–IDF

The intuition is to *down-weight* words that appear in many documents (because they discriminate poorly between documents) and *up-weight* words that appear only in a few (because their presence is informative).

Definition 2.2.4 (Inverse document frequency)**.** Given a corpus of N documents, the **document frequency** of a word w is the number of documents that contain w at least once,

$$\mathrm{df}(w) = \big|\{d \in \text{corpus} : w \in d\}\big|.$$

The **inverse document frequency** is

$$\mathrm{idf}(w) = \log \frac{N}{\mathrm{df}(w)}.$$

Common variants smooth the formula to avoid division by zero and dampen extreme values:

$$\mathrm{idf}_{\text{smooth}}(w) = \log\left(\frac{N+1}{\mathrm{df}(w)+1}\right) + 1.$$

Definition 2.2.5 (TF–IDF)**.** The **TF–IDF** weight of word w in document d is the product

$$\mathrm{tfidf}(w, d) = \mathrm{tf}(w, d) \times \mathrm{idf}(w).$$

Replacing each entry of the document–term matrix by the corresponding TF–IDF gives the TF–IDF document–term matrix.

Intuition.

- Words appearing in nearly every document have $\mathrm{df}(w) \approx N$, hence $\mathrm{idf}(w) \approx \log 1 = 0$. They are zeroed out — the same effect as stop-word removal, but data-driven.
- Words appearing in few documents have $\mathrm{df}(w) \ll N$, hence large $\mathrm{idf}(w)$. They retain (and amplify) their term-frequency contribution.
- Within a single document, TF–IDF still scales with raw term frequency, preserving emphasis on repeated key terms.

Property 2.2.1 (Boundedness of idf). For a corpus of N documents, $0 \le \mathrm{idf}(w) \le \log N$ when idf uses the unsmoothed formula. The maximum is reached when w appears in exactly one document, the minimum when w appears in all of them.

2.2.8 TF–IDF as a probabilistic measure

Remark 2.2.1 (Information-theoretic flavour). $\mathrm{idf}(w) = -\log \frac{\mathrm{df}(w)}{N}$ is the negative log-probability that a randomly chosen document contains the word w. A rare word "surprises" us when it appears, contributing many bits of information; a common word contributes few. This is a discrete analogue of pointwise mutual information, and the connection has been formalised: TF–IDF approximates a maximum-likelihood estimate under specific generative assumptions.

2.2.9 Worked examples

Example 1 (BoW from scratch — Easy). For the two-document corpus

- d_1: "*the cat sat on the mat*"
- d_2: "*the cat ate the fish*"

build the vocabulary, the document–term matrix with raw counts, and compute $\text{idf}(w)$ for each word.

Solution. $V = \{\text{the, cat, sat, on, mat, ate, fish}\}$.

	the	cat	sat	on	mat	ate	fish
d_1	2	1	1	1	1	0	0
d_2	2	1	0	0	0	1	1
df	2	2	1	1	1	1	1
idf	$\log 1 = 0$	0	$\log 2$	$\log 2$	$\log 2$	$\log 2$	$\log 2$

"*the*" and "*cat*" carry no IDF weight (they appear in every document), while *sat*, *on*, *mat*, *ate*, *fish* all carry weight $\log 2 \approx 0.69$.

Example 2 (Computing TF–IDF — Medium). A four-document corpus has $\text{df}(\textit{Tunisia}) = 1$ and $\text{df}(\textit{the}) = 4$. Document d_1 contains *Tunisia* once and *the* four times.

(a) Compute $\text{tfidf}(\textit{Tunisia}, d_1)$ and $\text{tfidf}(\textit{the}, d_1)$ using the unsmoothed formula.

(b) Comment on the result: which word's contribution dominates the document representation?

Solution. $\text{idf}(\textit{Tunisia}) = \log(4/1) = \log 4 \approx 1.39$. $\text{idf}(\textit{the}) = \log(4/4) = 0$. $\text{tfidf}(\textit{Tunisia}, d_1) = 1 \times 1.39 = 1.39$. $\text{tfidf}(\textit{the}, d_1) = 4 \times 0 = 0$. The discriminative word *Tunisia* dominates the representation, even though it appears only once. The function word *the*, which appears four times, is silenced. This is exactly the desired behaviour: a query for *Tunisia* should retrieve d_1, while a query for *the* should not.

Example 3 (Spam filtering with TF–IDF — Hard). Consider a spam-filtering corpus. The word *lottery* appears in 50 out of 5,000 training emails, all of which are spam. The word *meeting* appears in 1,200 emails, half spam, half legitimate. Argue, using TF–IDF and the discriminative power of each feature, why a logistic-regression classifier built on TF–IDF features will give a much larger weight to *lottery* than to *meeting*.

Solution. The IDF of *lottery* is $\log(5000/50) = \log 100 \approx 4.6$, while the IDF of *meeting* is $\log(5000/1200) \approx 1.4$. Already, after the IDF re-weighting, *lottery* contributes more to the representation than *meeting* per occurrence. Logistic regression then assigns weights to maximise log-likelihood; *lottery* is correlated with the spam label with probability 1, while *meeting* is correlated with the spam label with probability 0.5. The resulting weight on *lottery* reflects both the higher IDF amplification and the stronger label correlation. The classifier is therefore much more sensitive to the word *lottery* when scoring a new e-mail.

2.2.10 Python snippet: from text to TF–IDF in 5 lines

```
from sklearn.feature_extraction.text import CountVectorizer,
    TfidfVectorizer

corpus = [
```

```
    "the room was clean the room was quiet",
    "the breakfast was great the room was clean",
    "service was slow but the rooms were clean",
]

# Step 1 : raw counts
counts = CountVectorizer().fit_transform(corpus)
print("Raw counts shape:", counts.shape)
print("Raw counts dense:\n", counts.toarray())

# Step 2 : TF-IDF transform
tfidf = TfidfVectorizer().fit(corpus)
X = tfidf.transform(corpus)
print("\nVocabulary:", tfidf.get_feature_names_out())
print("TF-IDF matrix:\n", X.toarray().round(2))

# Inspect the IDF of each token
for term, score in zip(tfidf.get_feature_names_out(), tfidf.idf_):
    print(f"  idf('{term}') = {score:.3f}")
```

2.2.11 Exercises

1. Build the BoW vocabulary, the document–term matrix, the IDF vector and the full TF–IDF matrix (smoothed) for the corpus

   - d_1: "*NLP is fun and useful*"
   - d_2: "*Tunisia is a country in North Africa*"
   - d_3: "*NLP is widely used in Tunisia*"

2. Show that for a corpus of N documents, the maximum possible IDF (unsmoothed) is $\log N$ and is reached when a word appears in exactly one document. Conversely, show that words appearing in every document have $\text{idf}(w) = 0$ and contribute zero TF–IDF weight.

3. Two documents, d_1 and d_2, share exactly the same set of words but with different counts. Are their BoW vectors necessarily different? Are their normalised BoW vectors (length-normalised) necessarily different?

4. TF–IDF is purely lexical and shares no information across morphological variants (*run* and *running* are unrelated columns of the document–term matrix). Propose two engineering remedies (one classical, one modern) and one situation in which each is preferable.

5. (Open-ended.) Suppose you must build a search engine over a corpus of 100,000 Tunisian-Arabic news articles. Discuss whether you would prefer a TF–IDF index, a dense-vector index (Section **??**), or a hybrid of the two. Identify a concrete query that one method handles but not the other.